

Diagnóstico de CFDIGITAL — Estructura actual de la aplicación

Generado el 27/08/2026 · Tarea de diagnóstico únicamente (sin modificaciones de código).

1. Estructura actual de navegación

La barra (`src/components/layout/navbar.tsx`) tiene 6 enlaces: **Inicio, Dashboard, Propiedades, Leads, Publicaciones, Mi cuenta**. No existe menú "Multimedia" ni página `/login`.

El "login" no es una pantalla central: cada módulo (Propiedades, Leads, Publicaciones) renderiza su propio formulario de acceso inline (`properties-workbench.tsx:686` , `leads-workbench.tsx` , `publications-workbench.tsx:209`). El botón "Acceder" de la navbar lleva a `/properties` , que dispara ese login embebido. Registro y cuenta son páginas aparte (`/register` , `/account`).

La autenticación es real vía Supabase (`src/app/api/auth/*` : login, register, logout, session, refresh, profile, change-password, change-email).

2. Qué aparece en cada pantalla

Inicio (`src/app/page.tsx` + `home/home-overview.tsx`)

- Hero con imagen de departamento + título CFDIGITAL.
- Bloque de KPIs en tiempo real (totales de propiedades, publicadas, borrador, leads, multimedia, imágenes, videos, archivadas) leyendo `/api/dashboard/summary`.
- Tarjetas de acceso a los 4 módulos.

Dashboard (`dashboard/page.tsx` + `dashboard-summary.tsx`)

- Mismos 8 KPIs que Inicio, más tabla "Publicadas recientemente" (título, ciudad, fecha). Datos reales desde `/api/dashboard/summary`.

Propiedades (`properties/page.tsx` + `properties-workbench.tsx`)

- Formulario de alta/edición (título, tipo, operación, precio, recámaras/baños/estacionamiento, dirección, amenidades, requisitos de contratación, descripción).
- Filtros (búsqueda, estado, tipo, operación).
- Inventario con acciones: Editar, **Publicar** (cambia `status` de la propiedad), **Media**, **Contenido**, **Generar PDF**, Eliminar.
- Panel **Multimedia** por propiedad (subir imagen/video, marcar "procesada", eliminar).
- Panel **Contenido IA** por propiedad (generar copy por canal).

Multimedia

No es menú; es el sub-panel "Media" dentro de Propiedades (`media-panel` , líneas 972-1029). Sube `image` o `video` a Supabase Storage, muestra URL firmada, y permite cambiar un "estado" (`original` / `processed` / `edited` / `generated`). No hay editor ni procesamiento real: "Marcar procesada" solo cambia un flag en BD.

Leads (`leads/page.tsx` + `leads-workbench.tsx`)

CRM: alta de contactos, estados, recordatorios (uso real de `/api/leads`).

Publicaciones (`publications/page.tsx` + `publications-workbench.tsx`)

- Formulario para crear publicación (propiedad, canal, copia, hashtags, CTA, fecha opcional).
- Botón "Traer copia IA" (carga una generación ya creada en Propiedades).
- Panel "Scheduler" con botón "Ejecutar scheduler" y cola de jobs.
- Cola de publicaciones con Programar / Publicar ahora / Cancelar / Eliminar.

Ficha / PDF (`properties/[id]/ficha/page.tsx`)

Pantalla de detalle; el "PDF" es `window.print()` del navegador (impresión a PDF), no hay librería de PDF server-side.

3. Flujo actual para crear y publicar una propiedad

1. Login inline en Propiedades.
2. "Crear propiedad" → `POST /api/properties` (borrador).
3. En el inventario, botón **Media** → subir fotos/videos (`POST /api/properties/{id}/media` , a Supabase Storage).
4. Botón **Contenido** → "Generar copy" por canal (`POST /api/properties/{id}/generate-content`) → guarda en `ai_generations` . Provider por defecto = **mock**.
5. Desde Publicaciones: elegir propiedad + canal, "Traer copia IA" (lee `ai_generations`), crear borrador o programar (`/api/publications` + `/schedule`).
6. "Publicar ahora" → `publishPublicationNow` solo cambia `status` a `published` en BD. El scheduler (`/api/scheduler/run`) corre jobs vencidos y también los marca `published` .
7. "Generar PDF" abre la ficha e imprime.

No existe integración real con Facebook/Instagram/WhatsApp: "publicar" es solo un cambio de estado en la base de datos.

4. Dónde se suben fotos y cuántas permite

- En el panel Media de cada propiedad (`submitMedia` , línea 606). Endpoint `src/app/api/properties/[id]/media/route.ts` → `lib/services/property-media-service.ts` .
- Límites (`lib/validators/media.ts`): imagen ≤ 10 MB, video ≤ 200 MB. Tipos `image/*` y `video/*` .
- **No hay límite de cantidad:** puede subir tantas fotos/videos como quiera (solo restricción de tamaño por archivo). Se muestran con URL firmada (`signed_url`); las imágenes se renderizan en miniatura.

5. Staging, reel/video y PDF — cómo funcionan realmente

- **Staging:** no existe como funcionalidad. Lo más cercano es el campo `state` de cada media (`original` / `processed` / `edited` / `generated`) y el botón "Marcar procesada". Es solo un flag en BD; no hay área de previsualización/edición, ni "staging" de publicaciones.
- **Reel/video:** no hay generación. Solo se **suben** videos (≤200 MB) y se muestra un enlace "Ver video". No hay edición, recorte, ni creación de reel.
- **PDF:** `window.print()` en la ficha (`ficha/page.tsx:80`). Sin servidor de PDF.

6. Dónde se generan copias y para qué redes

- **Generación:** panel "Contenido IA" de Propiedades (`generateContent` , `content-panel` línea 1031). Canales: **Facebook, Instagram, WhatsApp** (`CONTENT_CHANNELS` en `lib/types/content.ts` y `properties-workbench.tsx:86`).
- Guarda en tabla `ai_generations` (`content-generation-service.ts`).
- **Reutilización:** en Publicaciones, "Traer copia IA" (`fetchAiCopy` , líneas 124-134) busca la generación existente para el canal seleccionado; si no existe, pide generarla antes desde Propiedades.
- El texto lo produce `lib/ai/providers/mock-provider.ts` (plantilla determinista). Hay stub de OpenAI (`openai-provider.ts`) pero requiere `AI_PROVIDER=openai` y `AI_API_KEY` (hoy vacíos en `.env` , así que **es simulación**).

7. Duplicidades y puntos confusos

- **Doble sentido de "Publicar":** en el inventario de Propiedades cambia el `status` de la propiedad (borrador→publicada); en Publicaciones significa "postear a red social". Mismo verbo, acciones distintas.
- **Copies en dos sitios:** se generan en Propiedades y se consumen en Publicaciones; el usuario debe saber el orden (generar primero en Propiedades) o "Traer copia IA" falla.
- **Login duplicado:** cada workbench tiene su propio formulario de login embebido; no hay página `/login` única ni estado global de sesión compartido entre módulos (aunque la cookie Sí es compartida).
- **Inicio vs Dashboard:** ambos muestran los mismos KPIs (HomeOverview ≈ DashboardSummaryView) — redundancia.
- **"Multimedia" prometido en textos** (hero, descripción) pero no es menú navegable; vive escondido dentro de Propiedades.

8. Funciones reales, incompletas y simuladas

Reales (funcionan con datos/BD)

- CRUD de propiedades, leads, publicaciones.
- Subida de multimedia a Supabase Storage (con URL firmada).
- Autenticación (login, registro, logout, cambio de contraseña/correo).
- Dashboard/KPIs y ficha con impresión a PDF.
- Validaciones de formularios y límites de archivo.

Incompletas / simuladas (interfaz o mock)

- **Copies de IA:** provider mock por defecto; no es LLM real salvo configurar OpenAI.
- **Publicación en redes sociales:** solo cambia estado en BD; no hay OAuth ni llamadas a APIs de FB/IG/WA.
- **Scheduler:** corre jobs y los marca publicados, pero no postea a ningún lado.
- **Staging / edición de media:** solo un flag de estado, sin herramienta.
- **Reel/video:** solo carga; sin generación ni edición.
- **PDF:** depende de la impresión del navegador (pierde estilos, no es automático).

9. Archivos o módulos involucrados

- Navegación/layout: `components/layout/navbar.tsx` , `app/layout.tsx` .
- Inicio/Dashboard: `app/page.tsx` , `home/home-overview.tsx` , `dashboard/page.tsx` , `dashboard/dashboard-summary.tsx` , `api/dashboard/summary/route.ts` .
- Propiedades/Alta/Media/Contenido: `properties/page.tsx` , `properties/properties-workbench.tsx` , `properties/[id]/ficha/page.tsx` , `api/properties/route.ts` , `api/properties/[id]/route.ts` , `api/properties/[id]/media/route.ts` , `api/properties/[id]/media/[mediaId]/route.ts` , `api/properties/[id]/generate-content/route.ts` , `api/properties/[id]/generations/route.ts` , `api/properties/ai-listing/route.ts` , `lib/services/property-service.ts` , `property-media-service.ts` , `content-generation-service.ts` , `lib/validators/media.ts` , `lib/types/media.ts` , `lib/types/content.ts` .
- IA: `lib/ai/factory.ts` , `providers/mock-provider.ts` , `providers/openai-provider.ts` , `ai-service.ts` .
- Publicaciones/Scheduler: `publications/page.tsx` , `publications/publications-workbench.tsx` , `api/publications/route.ts` , `api/publications/[id]/{schedule,publish,cancel}/route.ts` , `api/scheduler/jobs/route.ts` , `api/scheduler/run/route.ts` , `lib/services/publication-service.ts` , `scheduler-service.ts` .
- Leads: `leads/page.tsx` , `leads/leads-workbench.tsx` , `api/leads/*` .
- Auth/Cuenta: `api/auth/*` , `account/page.tsx` , `register/page.tsx` .
- Datos: `supabase/migrations/*` (properties, property_media, ai_generations, publications, scheduler_jobs, leads, auth.users).

10. Riesgos de modificar el flujo actual

- **"Publicar" con doble significado:** cualquier cambio en el botón de inventario de Propiedades afecta el `status` de la propiedad usado en filtros y dashboard; confundirlo con publicación social rompería reportes.
- **Acoplamiento ai_generations ↔ Publicaciones:** mover la generación de copies rompe el botón "Traer copia IA" si no se actualiza el canal/clave.
- **Provider mock marcado TEMPORAL:** `mock-provider.ts` y `factory.ts` dicen "debe reemplazarse". Cambiar a OpenAI real requiere `AI_API_KEY` y puede alterar el formato de copies y las pruebas (`mock-listing.test.ts` , `mock-ai-service.test.ts`).
- **URLs firmadas de media:** las imágenes usan `signed_url` (caducan); si se cambia la lógica de Storage, la galería puede dejar de mostrar fotos.
- **Scheduler depende de** `CRON_SECRET` y de un cliente service-role; modificar `run/route.ts` sin ese secreto rompe la ejecución automática.
- **Sin límite de cantidad de fotos:** subir muchas puede inflar Storage; cualquier límite nuevo rompería el validador actual (`media.ts`).
- **Sesión por cookie compartida:** los logins embebidos asumen la misma cookie; centralizar login o usar estado global puede requerir refactor de los 3 workbenches.